

“초등부 2번 / 중등부 1번. 가로등” 문제 풀이

작성자: 윤교준

부분문제 1

$x = 0$ 부터 $x = A_1$ 까지 정수 위치의 어두운 정도는 $0, 1, \dots, A_1$ 이다. 또한, $x = A_1 + 1$ 부터 $x = L$ 까지 정수 위치의 어두운 정도는 $1, 2, \dots, L - A_1$ 이다.

즉, 정렬된 어두운 정도 값에 간단한 규칙이 있으므로, A_1 과 $L - A_1$ 의 대소 비교 후 답을 출력할 수 있다.

부분문제 2

한 위치의 어두운 정도는 지문의 주어진 식에 따라 $\mathcal{O}(N)$ 에 계산할 수 있다.

따라서, 모든 위치의 어두운 정도를 계산한 후 정렬하면 $\mathcal{O}(NL)$ 의 시간 복잡도에 해결할 수 있다.

부분문제 3

가로등이 0부터 L 까지 등간격으로 떨어져 있다.

따라서, 출력한 수가 K 개가 될 때까지, 0을 N 번, 1 이상의 정수를 각각 $2(N - 1)$ 번씩 차례대로 출력하면 된다.

부분문제 4

다음을 관찰한다:

- 위치 $0 \leq x \leq A_1$ 의 어두운 정도는 $A_1 - x$.
- 위치 $A_i \leq x \leq A_{i+1}$ 의 어두운 정도는 $\min\{x - A_i, A_{i+1} - x\}$.
- 위치 $A_N \leq x \leq L$ 의 어두운 정도는 $x - A_N$.

각 구간에 대해, 각 위치의 어두운 정도는 $\mathcal{O}(1)$ 에 계산할 수 있다. 따라서, 모든 위치의 어두운 정도를 계산하여 정렬까지 $\mathcal{O}(L)$ 의 시간 복잡도에 수행할 수 있다.

부분문제 5

N 개의 가로등이 놓인 위치를 모두 출발지로 하여 BFS를 수행한다.

큐에서 (위치, 거리) 쌍을 제거할 때마다 거리 값을 출력하면, 그것이 문제에서 원하는 답이 된다.

큐에 삽입과 삭제는 총 $\mathcal{O}(N + K)$ 번 수행된다.

큐에 점을 삽입하기 전에 방문 여부를 검사해야 하며, 이는 `std::set` 등의 BBST 라이브러리를 사용하여 효율적으로 처리할 수 있다.

따라서, 총 시간 복잡도는 $\mathcal{O}((N + K) \lg(N + K))$ 이다.

$\mathcal{O}(N + K)$ 의 선형 시간과 공간 복잡도로도 문제를 해결할 수 있다.

“중등부 3번 / 고등부 2번. XOR 최대” 문제 풀이

작성자: 나정휘

부분문제 1

$N \leq 30$ 으로 입력이 매우 작으므로 S 에서 s_1 과 s_2 를 선택하는 $O(N^4)$ 가지 경우를 모두 확인하면 문제를 해결할 수 있다.

부분문제 2

두 가지 풀이를 설명한다.

풀이 1

음이 아닌 정수 N 개로 구성된 배열 A 에서 두 수를 골라 XOR한 값을 최대화하는 문제는 트라이(Trie)라는 자료구조를 이용해 $O(N \log(\max A_i))$ 에 해결할 수 있음이 잘 알려져 있다.

이 문제는 S 의 부분문자열 $O(N^2)$ 개 중 2개를 골라 XOR한 값을 최대화하는 문제이며, 각 부분문자열의 길이는 N 이하이므로 트라이(Trie)를 이용해 $O(N^3)$ 시간에 해결할 수 있다.

풀이 2

일반성을 잃지 않고 $|s_1| \geq |s_2|$ 라고 하자. 또한, S 에서 가장 먼저 등장하는 1의 위치를 x 라고 하자. 이때 인덱스는 1부터 시작한다.

관찰 1. s_1 이 S 의 접두어(prefix)인 경우만 생각해도 된다.

증명: s_1 이 S 의 접두어가 아닌 답이 있다고 가정하자. s_1 의 바로 앞 문자가 1이면 s_1 을 앞으로 한 칸 확장해서 값을 증가시킬 수 있으므로 모순이고, s_1 의 바로 앞 글자가 0이면 앞으로 한 칸 확장해도 답이 변하지 않는다. 따라서 s_1 이 S 가 접두어가 아니라면 접두어가 될 때까지 s_1 을 확장할 수 있다.

관찰 2. 두 부분문자열을 XOR해서 뒤에서부터 $N - x + 1$ 번째 비트보다 더 높은 비트를 켤 수 없다.

증명: $t \geq N - x + 2$ 번째 비트를 켜기 위해서는 $S[1 \cdots x - 1]$ 사이에 1이 존재해야 하지만, x 의 정의에 의해 $S[x]$ 보다 앞에 1이 존재할 수 없다.

관찰 3. s_1 은 $S[x \cdots N]$ 을 포함하는 형태만 고려해도 된다.

증명: s_1 이 $S[x \cdots N]$ 을 포함하지 않으면 $N - x + 1$ 번째 비트를 켜지 못함을 보이면 되고, 관찰 2와 비슷하게 증명할 수 있다.

세 가지 관찰을 종합하면 다음과 같은 결론을 얻을 수 있다.

정리 1. $s_1 = S$ 인 정답이 존재한다.

증명: 관찰 3에 의해 s_1 이 S 의 접미어(suffix)인 정답이 존재한다. 또한, 관찰 1에 의해 s_1 을 앞으로 확장한 더라도 답이 감소하지 않는다. 따라서 $s_1 = S$ 인 정답이 존재한다.

S 의 모든 부분문자열 s_2 를 보면서, S 와 s_2 를 XOR한 값 중 최댓값을 구하면 된다. 이 경우 s_2 로 가능한 부분문자열이 $O(N^2)$ 가지이고 XOR 연산을 수행하는 데 $O(N)$ 시간이 걸린다. 따라서 $O(N^3)$ 시간에 문제를 해결할 수 있다.

부분문제 3

두 가지 풀이를 설명한다.

풀이 1

부분문제 2의 두 번째 풀이를 트라이(Trie)를 이용해 최적화하면 $O(N^2)$ 시간에 해결할 수 있다.

풀이 2

편의상 S 에 0과 1이 모두 1개 이상 등장하는 경우만 생각하자. S 에 1이 등장하지 않으면 정답은 0이고, 0이 등장하지 않으면 정답은 S 의 마지막 비트를 0으로 바꾼 것이다.

S 에서 가장 처음 등장하는 1과 연속한 1의 개수를 C , 그 바로 뒤에 등장하는 0의 위치를 P 라고 하자.

관찰 4. s_2 의 첫 글자를 $S = s_1$ 의 P 번째 글자와 XOR 연산하는 정답이 존재한다.

증명: $S[P]$ 는 0이고 그 앞($S[P-1]$)에 1이 있으므로 XOR한 결과의 P 번째 비트를 1로 만드는 방법이 존재하며, 그 위치를 1로 만들지 않고 더 좋은 답을 만들 수 없다. 또한, s_2 의 첫 글자가 $S[P]$ 가 아닌 그보다 더 앞에 있는 문자와 매칭되더라도 답이 증가하지 않음은 쉽게 알 수 있다.

이 관찰을 이용하면 s_2 의 후보를 C 개로 줄일 수 있다. 예를 들어 $S = 011110ABCDE\dots$ 형태라면, S 와 s_2 를 매칭하는 방법은 아래 네 가지 뿐이다.

```
S = 011110ABCDE...
s2 =      11110A...
s2 =      1110AB...
s2 =      110ABC...
s2 =      10ABCD...
```

이와 같이 s_2 의 후보는 $C(\leq N)$ 개밖에 존재하지 않는다. 실제로 고려해야 하는 s_2 가 $O(N)$ 가지이므로 $O(N^2)$ 시간에 정답을 구할 수 있다.

부분문제 4

부분문제 3의 두 번째 풀이처럼 $S[P\dots N]$ 과 XOR한 값이 최대가 되는 s_2 를 찾는 방식으로 해결할 것이다.

A 가 주어졌을 때 $A \oplus B$ 를 최대화하는 B 를 찾는 것은 $\text{not}A \oplus B$ 를 최소화하는 B 를 찾는 것과 같다. 이러한 B 를 찾는 문제는 가능한 B 의 후보들 중 $\text{not}A$ 보다 작거나 같으면서 가장 큰 수, 또는 $\text{not}A$ 보다 크거나 같으면서 가장 작은 수 중 하나가 정답이다. 따라서 부분문자열의 사전 순 비교를 $O(T(N))$ 시간에 할 수 있으면 전체 문제를 전처리 시간을 제외하고 $O(N \times T(N))$ 시간에 해결할 수 있다.

문자열 $A \# \text{not}A$ 의 접미사 배열(Suffix Array)를 사용하면 전처리를 $O(N \log N)$, 부분문자열의 사전 순 비교를 $O(1)$ 에 할 수 있고, 문자열 해시와 이분 탐색을 사용하면 전처리를 $O(N)$, 부분문자열의 사전 순 비교를 $O(\log N)$ 에 할 수 있다. 두 방법 모두 전체 시간 복잡도는 $O(N \log N)$ 이다.

부분문제 5

부분문제 3의 두 번째 풀이를 최적화할 것이다. $S = 011110ABCDE\dots$ 형태라고 가정하자.

$S = 011110ABCDE\dots$

$s_2 = 11110A\dots$

$s_2 = 1110AB\dots$

$s_2 = 110ABC\dots$

$s_2 = 10ABCD\dots$

만약 $A = 1$ 이면 $s_2 = 10ABCD$ 로 확정할 수 있고, $A \neq 1, B = 1$ 이면 $s_2 = 110ABC$ 로 확정할 수 있다. 이와 같이 P 이후로 처음 나오는 1의 위치를 $P + i$ 라고 하면, s_2 는 $P - \min(C, i)$ 에서 시작하는 문자열로 확정할 수 있다. 따라서 정답이 되는 s_2 를 $O(N)$ 에 구할 수 있어 전체 문제를 $O(N)$ 시간에 해결할 수 있다.

“중등부 4번 / 고등부 3번. 트리 뽑아내기” 문제 풀이

작성자: 이은조

부분문제 1

문제에 주어진 연산을 그대로 구현하면 $O(N^2)$ 의 시간복잡도로 문제를 해결할 수 있다.

부분문제 2

최솟값, 최댓값을 관리하는 자료구조인 힙(Heap)과 같은 원리로 뽑아내기 연산 이후에도 모든 $2 \leq i \leq N$ 에 대해 $A_{P_i} < A_i$ 가 성립한다는 것을 증명할 수 있다. 그러므로 루트 정점은 항상 모든 정점들 중 가장 작은 가중치를 갖게 된다. 그렇기 때문에 답이 항상 $1, 2, \dots, N$ 이다.

부분문제 3

어떤 정점 u 의 자식들 $v_1, \dots, v_k$ 가 있을 때, 일반성을 잃지 않고 $A_{v_1} < \dots < A_{v_k}$ 라고 하자. u 가 특별한 경로로 선택된다면 가장 작은 가중치를 가진 v_1 도 특별한 경로로 선택될 것이다. 부분문제 조건에 의해 뽑아내기 연산 이후에 v_1 정점의 가중치는 더욱 작아질 것이므로, 여전히 v_1 이 특별한 경로로 선택됨을 알 수 있다. 즉, v_1 을 루트로 하는 서브트리에 속한 정점들이 모두 제거될 때까지 v_1 이 특별한 경로로 선택된다. 즉, $v_1, \dots, v_k$ 순으로 그 정점의 서브트리에 있는 모든 정점들이 차례대로 선택된다. 그러므로 루트 정점에서부터 DFS를 수행하되 가중치가 작은 정점부터 방문하면 DFS에서의 정점 방문 순서가 답이라는 것을 알 수 있다.

부분문제 4

차수가 3 이상이거나 리프인 정점들만 고려하면 압축된 형태의 트리를 만들 수 있다. 이렇게 만든 압축된 트리의 간선에는 여러 개의 정점들이 줄지어 달려 있는 형태가 될 것이다. 압축된 트리의 각 정점마다 인접한 정점들의 가중치를 우선순위 큐로 관리하고, 압축된 트리의 간선에 줄지어 달려있는 정점들은 큐로 관리하자. 문제 조건에 의해 압축된 트리의 높이가 작으므로 특별한 경로를 $O(20 \log N)$ 의 시간복잡도로 찾을 수 있다. 특별한 경로를 따라 올라가면서 간선의 큐와 정점의 우선순위 큐를 갱신해주면 뽑아내기 연산 역시 $O(20 \log N)$ 시간복잡도로 구현할 수 있다. 총 시간복잡도는 $O(20N \log N)$ 이다.

부분문제 5

서브트리에서 부분문제를 해결하고, 이를 합쳐서 부모에 대한 문제를 해결하는 방식으로 해결하자. 어떤 정점 u 의 자식들 $v_1, \dots, v_k$ 가 있을 때, v_i 를 루트로 하는 서브트리의 답이 수열 $B_{v_i}[\dots]$ 라고 하자. 이는 v_i 가 특별한 경로에 포함되어 뽑아내기 연산을 수행했을 때 v_i 정점의 가중치가 수열 $B_{v_i}[\dots]$ 순서대로 변화한다는 의미이다. 우리는 이 정보를 이용해서 u 를 루트로 하는 서브트리의 답 수열 $B_u[\dots]$ 를 구해야 한다. 자명히 $B_u[\dots]$ 의 첫 번째 수는 A_u 이다. 이제 나머지 수들은 병합 정렬을 수행하듯이 $B_{v_1}[\dots], \dots, B_{v_k}[\dots]$ 중 가장 작은 수를 뽑아 $B_u[\dots]$ 의 끝에 추가해주는 것을 반복하면 모두 구할 수 있다.

이를 단순히 구현하면 시간 초과를 받기 때문에 최적화가 필요하다. 수열 $B_i[\dots]$ 을 앞에서부터 봤을 때 최댓값이 갱신되는 순간 새로운 그룹으로 묶는 방식으로 그룹을 나누어 관리하고, 그룹의 대푯값을 그룹의 맨 왼쪽 수로 정의하자. 예를 들어 $2, 1, 3, 5, 4, 8, 6, 7$ 은 $[2, 1], [3], [5, 4], [8, 6, 7]$ 로 관리하는 것이다. 이렇게 그룹을 나누면 여러 수열을 하나로 합칠 때 그룹으로 묶인 수들은 항상 그룹으로 묶여 있고, 그룹의 대푯값

순서대로 합쳐진다는 것을 알 수 있다. 또한 첫 번째 수보다 대푯값이 작은 그룹들은 하나의 새로운 그룹으로 묶이게 된다. 그러므로 그룹들을 우선순위 큐로 관리하고, 작은 집합에서 큰 집합으로 합치는 테크닉을 활용하면 $O(N \log^2 N)$ 의 시간복잡도로 문제를 해결할 수 있다.

조금 더 생각해보면, 결국 이 풀이에서 구하는 것이 부모-자식 순서 관계가 유지되는 순열 중 사전순으로 가장 앞서는 순열이라는 것을 귀납적으로 증명할 수 있다. 그러므로 우선순위 큐를 사용해서 루트 정점에서부터 가장 가중치가 작은 정점을 선택해나가는 방식으로 $O(N \log N)$ 에 문제를 해결할 수 있다.

“고등부 4번. 점수 경주” 문제 풀이

작성자: 이종영

부분문제 1

어떤 정점 v 를 지나가는 모든 참가자들은 v 까지 같은 전략을 취함을 이용하자. 최적 전략을 DFS로 구현하면서 현재 S 와 C 를 관리하면 $O(N^2)$ 에 문제를 해결할 수 있다.

부분문제 5

각 정점 u 에 대해, $v \leq u$ 인 v 들을 처리하자. $v > u$ 인 v 들은 뒤집은 후 같은 방법으로 처리할 수 있다.

1번 정점에서 u 까지의 거리를 D_u 라고 하자. $D_u - D_v < 0$ 인 마지막 v 에 대해, v 를 지나는 참가자들은 v 에서 점수를 0으로 만든다. 또한 v 를 지나기 전에는 점수를 0으로 만들지 않는다. 이를 이용하면 v 의 답에서 u 의 답을 얻을 수 있으며, 동적 프로그래밍을 통해 답을 구할 수 있다. v 를 구하는 것은 1차 대회 "오름차순" 문제와 유사하게 monotone stack을 관리하면 된다.

부분문제 6

센트로이드 분할을 이용해 문제를 해결한다. 센트로이드 c 에 대해, c 를 지나는 모든 경로들을 처리한 후, c 를 제거하고 나뉜 각각의 트리에서 다시 문제를 해결하는 분할 정복 방식을 사용한다.

c 를 기준으로 $x_1, \dots, x_k$ 의 서브트리로 나뉜다고 하자. x_1 의 서브트리의 속하는 u 에 대한 답을 구해보자. 각 v 에 대해 $u \rightarrow c$ 경로와, $c \rightarrow v$ 경로로 분해해서 생각하자.

$u \rightarrow c$ 경로에서 c 에 도달했을 때의 점수 및 점수를 0으로 바꾼 횟수는 항상 일정하다. 이는 부분문제 5와 유사하게 동적 프로그래밍을 통해 구할 수 있는데, 이 서브태스크의 경우 트리의 높이가 낮으므로 점수가 0 미만이 되는 최초 지점을 부모를 타고 올라가면서 구해주면 충분하다. 이제 결과 C 값은 모든 v 에 대해 더해지므로, 트리의 크기 M , 서브트리의 크기 sz 에 대해 $M - sz_{x_1}$ 만큼 곱해서 답에 더해준다.

$c \rightarrow v$ 의 상의 정점 w 에 대해, w 에서 점수가 0 미만이 되는 경우를 생각해보자. c 에서 점수가 0인 경우를 기준으로, 점수가 0 이상인 정점들은 c 에서의 점수가 더 높아도 점수가 0 미만이 되지 않는다. 즉, c 에서 점수가 0인 경우 점수가 0 미만이 되는 정점들이 w 의 후보가 된다. 그리고 어떤 v 에 대한 답은 마지막 w 부터의 거리이다. 이러한 w 들은 c 에서의 점수가 높아질수록 c 에서 가까운 부분부터 점점 점수가 0 이상이 되게 된다. 구체적으로, c 와 거리 D 에 대해 c 에서의 점수가 $-D$ 이상인 경우 점수가 0 이상이 된다.

두 경우를 종합하면 각 u 에 대해 c 에서의 점수를 구한 후, 해당 점수에 대한 S 값과 C 값을 스위핑을 통해 구할 수 있음을 알 수 있다. 시간복잡도는 $O(N \log^2 N)$ 이다.

부분문제 8

$u \rightarrow c$ 경로의 값을 빠르게 구해야 한다. 세그먼트 트리에서의 이분 탐색, 스패스 테이블, 스택을 관리하고 pop할 정점들을 이분탐색으로 구하고 복원 연산을 구현하는 방법 등으로 $O(\log N)$ 에 구할 수 있다. 정해는 마지막 방법을 이용하여 구현되었다. 시간복잡도는 $O(N \log^2 N)$ 이다.